React

Co to je?

- Je open-source frontend JavaScriptová knihovna
- Vytvořena společností Meta
- **Je to knihovna, ne framework:**
  - Narozdíl od kompletních frameworků se soustředí na jednu specifickou oblast - UI
- Používá se s dalšími knihovnami
- Používá se především pro tvorbu SPA (Single Page Application)
  - Aplikace které běží na jedné HTML stránce a obsah se dynamicky mění bez nutnosti načítat celou stránku znovu
- **Hlavní vlastnosti:**
  - **Komponentový přístup:**
    - Celé UI se skládá z malých, znovupoužitelných komponent
    - Komponenty poté pak mají své vstupní vlastnosti (props) a svůj vnitřní stav (state)
  - **Virtuální DOM:**
    - Je reprezentace skutečného DOMu uložená v paměti
    - Místo toho, aby React pracoval přímo se skutečným DOMem (což může být pomalé a výkonnostně náročné), vytváří si kopii UI komponent
    - Tato virtuální kopie je lehký JavaScriptový objekt, který zrcadlí strukturu skutečného DOMu
  - **JSX:**
    - JSX -> JavaScript + XML
    - Umožňuje psát HTML kód přímo do JavaScriptu

```
const myElement = <h1>I Love JSX!</h1>;

createRoot(document.getElementById('root')).render(
  myElement
);
```

JSX

```
const myElement = React.createElement('h1', {}, 'I do not use JSX!');

createRoot(document.getElementById('root')).render(
  myElement
);
```

Bez JSX

Instalace

- **Pomocí npm muzem vytvořit react projekt**
  - **npm create vite@latest muj-projekt — —template react**
- Tím se vytvoří nový projekt, nainstalují se závislosti a spustí se vývojový server

Základní komponenty a props

- **Příklad jednoduché komponenty bez props:**

```
function Pozdrav() {
    return <h1>Ahoj svete</h1>
}

export default Pozdrav
```

  - **A následně v App.jsx napíšeme toto:**

```
function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <Pozdrav />
    </>
  )
}
```

```
import Pozdrav from "./pozdrav"
```

- **Příklad komponenty s props:**

```
function Karta({ jmeno, vek }) {
    return (
      <div>
        <h2>{jmeno}</h2>
        <p>Věk: {vek}</p>
      </div>
    )
  }

  export default Karta
```

  - **V App.jsx zase napíšeme import a do return:**

```
return (
  <>
    <Pozdrav />
    <Karta jmeno="Jana" vek={22}/>
  </>
)
```

React hooks

- Jsou speciální funkce, které přidávají komponentám schopnosti jako správa stavu nebo práce s vedlejšími efekty
- **Pravidla hooků:**
  - Mohou být volány pouze uvnitř React komponent
  - Mohou být volány pouze na nejvyšší úrovni komponenty (ne uvnitř podmínek, cyklů nebo vnořených funkcí)
- **useState:**
  - Nejzákladnější hook pro práci se stavem
  - **const [hodnota, setHodnota] = useState(počátečníHodnota)**
    - **hodnota** -> aktuální stav
    - **setHodnota** -> funkce pro změnu stavu
    - Při každém zavolání **setHodnota** se komponenta překreslí s novou hodnotou
  - **Příklad**:

```
import { useState } from "react"

export default function Counter({ nazev }) {
  const [count, setCount] = useState(0)

  return (
    <div>
      <h2>{nazev}</h2>
      <p>Počet: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(count - 1)}>-1</button>
    </div>
  )
}
```

```
function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <Counter nazev="CA" />
      <Counter nazev="CB" />

    </>
  )
}
```

    - Každá instance **Counter** má vlastní state. Když klikneme na +1 u CA, CB se nezmění. Každá komponenta si drží count nezávisle

- **useEffect**:
  - Slouží k provádění „vedlejších efektů“, tedy věcí, které nesouvisí přímo s vykreslováním UI např.:
    - Načítání dat z API
    - Nastavení timeru
    - Změna titulku stránky
    - ……
    - **Příklad**:

```
import { useState, useEffect } from 'react'

function App() {
  const [count, setCount] = useState(0)

  useEffect(() => {
    document.title = `Počet: ${count}`
  }, [count])

  return (
    <>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </>
  )
}
export default App
```

  - Pokaždé když se změní count, **useEffect** se spustí a aktualizuje titulek záložky v prohlížeči

- **useContext**:
  - Řeší problém zvaný prop drilling (situace, kdy potřebujeme předat data přes mnoho vrstev komponent)
    - App -> Layout -> Sidebar -> Menu -> MenuItem
    - Bez toho by jsme museli předávat props přes každou úroveň, i když jen potřebujeme jen MenuItem. S **useContext** si MenuItem data vezme přímo
  - **Příklad**:

- **createContext** - vytvoří se context
- **Provider** - obalí se tím komponenta a předá se value
- **useContext** - kdekoliv uvnitř Provideru se hodnota vyzvedne
- Bez kontextu by jsme museli psát **<Navbar user=„Honza“ />** a kdyby mezi App a Nav bylo pět komponent , museli bychom props předávat přes každou z nich

```
import { createContext, useContext } from "react"

const UserContext = createContext(null)

function Navbar() {
  const user = useContext(UserContext)
  return <p>Přihlášen: {user}</p>
}

function App() {
  return (
    <UserContext.Provider value="Honza">
      <Navbar />
    </UserContext.Provider>
  )
}
export default App
```

React Router

- Umožňuje vytvářet navigaci mezi „stránkami“ v SPA aplikaci
- Stránka se ve skutečnosti nenačítá znovu
- React Router jen vyměňuje komponenty podle URL adresy
- **Instalace:**
  - **npm install react-router-dom**
- **Příklad**:

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom"
import Domov from "./Domov"
import ONas from "./ONas"
import Kontakt from "./Kontakt"

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Domů</Link>
        <Link to="/o-nas">O nás</Link>
        <Link to="/kontakt">Kontakt</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Domov />} />
        <Route path="/o-nas" element={<ONas />} />
        <Route path="/kontakt" element={<Kontakt />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App
```

- **BrowserRouter** - obaluje celou aplikaci a umožňuje routing
- **Link** - funguje jako <a> tag, ale stránku nenačítá znovu, jen mění URL a vymění komponentu
- **Routes** - obaluje všechny routy
- **Route** - definuje, jaká komponenta se zobrazí na jaké URL

- **Rozdíl oproti klasickému HTML:**
  - V klasickém webu kliknutí na odkaz pošle nový požadavek na server a načte celou stránku
  - React Router jen změní URL v prohlížeči a vyměňuje komponenty na klientské straně